Jordan University of Science and Technology

Computer Engineering Department

CPE 473: Operating Systems

Programming Assignment #2: Threading

Notes:

1- **The due date is 21/12/2024 at 11:55 PM.**
2- **Late submissions (i.e. after the due date) will not be accepted.**
3- **Submit the source code files and the PDF report. Write your name and ID at the top**
4- **Groups are required (3 students; it is OK to be from different sections)**
5- **Only one submission needed per group. All group members must attend the discussion.**

In this assignment, you will implement a multi-threaded program (using C/C++) that will check for **Prime Numbers and Palindrome Numbers in a range of numbers**. Palindrome numbers are numbers that their decimal representation can be read from left to right and from right to left (e.g. 12321, 5995, 1234321). The program will create T worker threads to check for prime and palindrome numbers in the given range (T will be passed to the program with the Linux command line). Each of the threads works on a part of the numbers within the range.

Your program should have some global shared variables: **numOfPrimes**, which will track the total number of prime numbers found by all threads. **numOfPalindroms**, which will track the total number of palindrome numbers found by all threads. **numOfPalindromicPrimes**, which will count the numbers that are BOTH prime and palindrome found by all threads. **TotalNums**: which will count all the processed numbers in the range. In addition, you need to have arrays (PrimeList, PalindromeList, PalindromicPrimesList) which will record all the founded numbers from the 3 categories.

When any of the threads starts executing, it will print its number (0 to T-1), and then the range of numbers that it is operating on. When all threads are done, the main thread will print the total count of numbers found from each of the 3 categories, in addition to printing all these numbers.

You should write two versions of the program: The first one doesn't consider race conditions, and the other one is thread-safe. The input will be provided in an input file (in.txt), and the output should be printed to an output file (out.txt). The number of worker threads will be passed through the command line, as mentioned earlier.

The input will be read from a file (in.txt) and will simply have two numbers rangeStart and rangeEnd, which are the beginning and end of the numbers to check for numbers from each category, inclusive. The list of prime numbers will be written to the output file (out.txt), all the other output lines (e.g. prints from threads) will be printed to the standard output terminal (STDOUT).

Grading Rules:

- Each student is expected to fully understand all the aspects and details of the entire code.
- Only one eLearning (Moodle) submission per group, but all students must attend the discussion.
- If you had any technical issues with submitting through eLearning, you need to send your submission to us (i.e. Dr. Mohammad and Eng. Hanadi) through email BEFORE the deadline.
- The grading will take place in a Linux environment (Ubuntu, using the g++ compiler).
- You MUST implement the code using this same environment to get a full mark.
- Your grade will depend on multiple aspects, such as:
    1. The correctness of your code's output across multiple secret test cases
    2. Your code's ability to handle corner test cases like having many processes or large processing times.
    3. Your answers to the questions during the discussion.

Tasks:

In this assignment, **you will submit your source code files for the thread-safe and thread-unsafe versions, in addition to a report (PDF file).** The report should show the following:

1. Screenshot of the main code
2. Screenshot of the thread function(s)
3. Screenshot highlighting the parts of the code that were added to make the code thread-safe, with explanations on the need for them
4. Screenshot of the output of the two versions of your code (thread-safe vs. non-thread-safe), when running passing the following number of threads (T): 1, 4, 16, 64, 256, 1024.
5. Based on your code, how many computing units (e.g. cores, hyper-threads) does your machine have? Provide screenshots of how you arrived at this conclusion, and a screenshot of the actual properties of your machine to validate your conclusion. It is OK if your conclusion doesn't match the actual properties, as long as your conclusion is reasonable.

Hints:

1. **Read this document carefully multiple times to make sure you understand it well. Do you still have more questions? Ask me during my office hours, I'll be happy to help!**
2. To learn more about prime and palindrome numbers, look at resources over the internet (e.g. link1, link2, link3) . We only need the parts related to the simple case, no need to implement any optimizations.
3. Plan well before coding. Especially on how to divide the range over worker threads. How to synchronize accessing the variables/lists.
4. For passing the number of threads (T) to the code, you will need to use *argc*, and *argv* as parameters for the main function. For example, the Linux command for running your code with two worker threads (i.e. T=4) will be something like: "./a.out 4"
5. The number of threads (T) and the length of the range can be any number (i.e. not necessarily a power of 2). Your code should try to achieve as much *load balancing* as possible between threads.

6. For answering Task #5 regarding the number of computing units (e.g. cores, hyper-threads) in your machine, search about “diminishing returns”. You also might need to use the Linux command “*time*” while answering Task #4, and use input with range of large numbers (e.g. millions).
7. You will, obviously, need to use pthread library and Linux. I suggest you use the threads coding slides to help you with the syntax of the pthread library

Sample Input (in.txt), assuming passing T=4 in the command line:

700666000 700666032

Sample Output (STDOUT terminal part):

ThreadID=0, startNum=700666000, endNum=700666008

ThreadID=1, startNum=700666008, endNum=700666016

ThreadID=2, startNum=700666016, endNum=700666024

ThreadID=3, startNum=700666024, endNum=700666032

totalNums=32, numOfPrime=3, numOfPalindrome=1, numOfPalindromicPrime=1

Sample Output (out.txt part):

The prime numbers are:

700666007

700666009

700666019

The palindrome numbers are:

700666007

The palindromicPrime numbers are:

700666007